

Report: Distributed Systems Homework 2

Moida Praneeth Jain (2022101093), Richa Verma (2024202010)

September 12, 2025

1 MPI Sparse Matrix Row-Row Multiplication

1.1 Problem Understanding

The goal of this project is to implement a parallel sparse matrix-matrix multiplication (SpGEMM) algorithm using the Message Passing Interface (MPI). The multiplication computes $C = A \times B$, where A and B are large, sparse matrices. A row-oriented approach is used, where rows of matrix A are distributed among processes to compute corresponding rows of the result matrix C. The implementation is efficient, scaling with the number of available processor cores and handling the sparse data structures effectively.

1.2 System Setup

- **Programming Language:** C++ (Standard: C++17)
- **Parallelism Framework:** OpenMPI
- **Cluster Configuration:** The performance analysis was conducted on the SLURM cluster, utilizing a single node with 32 cores and 93Gi RAM.
- **Compilation:** The code was compiled using `mpic++` with `-O3` and `-march=native` flags for performance optimization.

1.3 Design and Implementation

The implementation follows a primary-worker (master-slave) pattern, where rank 0 is the primary process responsible for data distribution and result aggregation.

1.3.1 Parallelization Strategy

The core strategy is to parallelize the computation of the output matrix C by distributing the rows of matrix A among the available MPI processes. Each process p is responsible for calculating a subset of rows of C, denoted C_p , by multiplying its assigned rows of A, denoted A_p , with the entire matrix B.

1.3.2 Data Distribution

1. **Initial Read:** The primary process (rank 0) reads the entire sparse matrices A and B from an input file. Matrix B is read into a memory-efficient, CSR-like format (using three separate vectors for row pointers, column indices, and values).
2. **Matrix B Broadcast:** The complete matrix B is broadcast to all worker processes using `MPI_Bcast`. This is necessary because every process needs full access to B to compute its assigned rows of C.
3. **Matrix A Distribution (Dynamic Load Balancing):** To distribute the rows of matrix A, a dynamic load balancing scheme is used. Rank 0 iterates through each row of A and assigns it to the worker process that currently has the minimum accumulated workload. The workload is estimated by the total number of non-zero elements in the rows assigned to that worker so far. This helps to evenly

distribute the computational effort, especially when the rows of A have varying sparsity. The row data is then transmitted to the respective workers using a combination of `MPI_Scatter` (for counts) and point-to-point `MPI_Send/MPI_Recv` calls.

1.3.3 Communication Patterns

- **MPI_Bcast:** Used to efficiently distribute matrix dimensions (N, M, P) and the entirety of matrix B to all processes.
- **MPI_Scatter** and **MPI_Send/MPI_Recv:** Used in combination to distribute the rows of matrix A from rank 0 to the workers.
- **MPI_Reduce:** Used at the end of the computation to find the maximum execution time among all processes for accurate benchmarking.
- **MPI_Send/MPI_Recv** (for results): After computing their local results, worker processes send their calculated rows of C back to the primary process (rank 0), which assembles the final output matrix.

1.3.4 Computational Optimization

A hybrid approach is used for the inner multiplication loop. Based on an estimated number of floating-point operations, the algorithm chooses between two accumulator strategies:

- **Hash Map** (`std::unordered_map`): For highly sparse result rows, a hash map is used to accumulate the values for each column index. This is memory-efficient when the number of non-zero elements in the output row is small.
- **Dense Vector** (`std::vector`): If the estimated number of operations exceeds a threshold (`DENSE_THR`), a dense vector is used as an accumulator. This is faster for denser output rows as it avoids the overhead of hash collisions and lookups.

1.4 Execution Details

- **Compilation Command:** The project is compiled using the MPI C++ wrapper with performance optimizations.

```
mpic++ -O3 -march=native -std=c++17 main.cpp -o sparse_matmul_cpp
```

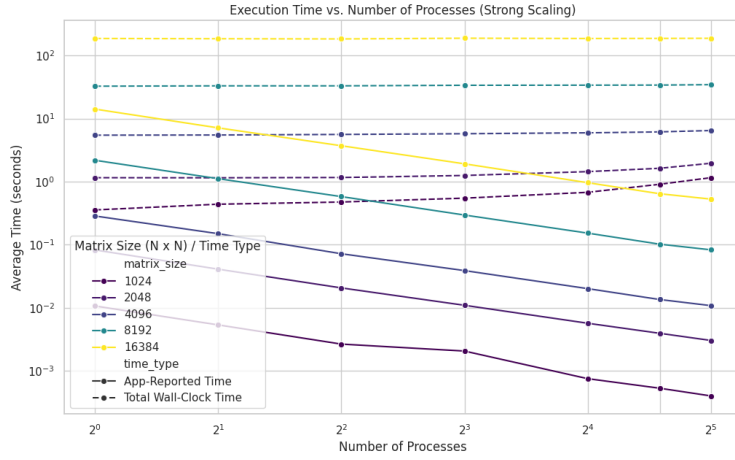
- **Execution Command:** The program is run using `mpirun`, specifying the number of processes and the input file.

```
mpirun -n <num_procs> --oversubscribe ./sparse_matmul_cpp <input_file>
```

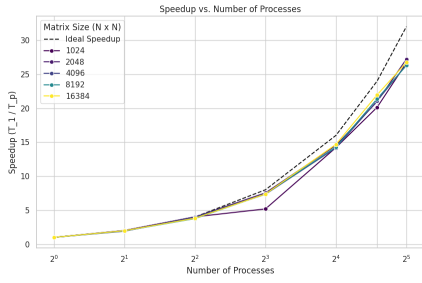
- `<num_procs>`: The total number of MPI processes to launch.
- `<input_file>`: Path to the text file containing the sparse matrices A and B.

1.5 Performance Evaluation

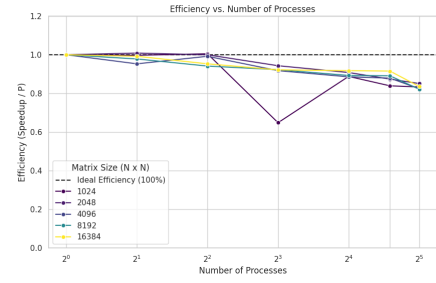
The performance was benchmarked by running the program with varying numbers of processor cores on matrices of different sizes. The following plots were generated from the `slurm_scaling_results.csv` data.



(a) Execution Time



(b) Speedup



(c) Efficiency

Figure 1: Scaling Analysis: (a) Execution time (b) Speedup (c) Efficiency vs. number of cores

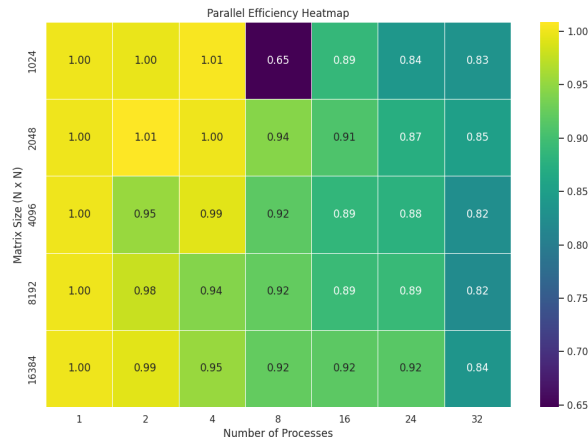


Figure 2: Efficiency Heatmap. This heatmap visualizes the parallel efficiency across different matrix sizes and core counts.

1.6 Challenges and Limitations

- **Rank 0 Bottleneck:** The primary process (rank 0) is responsible for all initial I/O and data distribution. This centralized approach can become a bottleneck, as all other processes must wait for rank 0 to finish reading and sending data before computation can begin.
- **Memory Usage:** Broadcasting the entire matrix B to all processes consumes a significant amount of memory on each node, limiting the maximum problem size that can be solved.
- **Result Aggregation:** All result rows are sent back to rank 0. For very large matrices, this can overwhelm the memory of the primary process and create a communication bottleneck at the end of the computation.

1.7 Conclusion and Future Work

The MPI-based sparse matrix multiplication implementation successfully parallelizes the computation, achieving good speedup and scalability. The dynamic load balancing scheme for distributing rows of matrix A is effective.

Future improvements could include:

- Implementing parallel I/O to remove the rank 0 bottleneck for data loading.
- Exploring a 2D decomposition of the matrix to partition both matrix A and B, reducing the memory footprint on each node.
- Using a tree-based communication pattern for aggregating results instead of sending everything directly to rank 0.

2 MapReduce K-Means Clustering

2.1 Problem Understanding

This project implements the K-Means clustering algorithm using a MapReduce paradigm simulated with MPI. K-Means is an iterative algorithm that partitions a dataset into K distinct, non-overlapping clusters. The MapReduce model is a natural fit for this: the **Map** phase assigns each data point to its nearest cluster centroid, and the **Reduce** phase recalculates the new centroids based on the mean of the points assigned to them.

2.2 System Setup

- **Programming Language:** Rust
- **Parallelism Framework:** OpenMPI (used to simulate the MapReduce model)
- **Cluster Configuration:** The performance analysis was conducted on the SLURM cluster, utilizing a single node with 32 cores and 93Gi RAM.
- **Build System:** Cargo, with the project built in release mode for optimization.

2.3 Design and Implementation

The implementation simulates the MapReduce pattern over several iterations until the cluster centroids converge or a maximum number of iterations is reached. Each iteration consists of a Map phase and a Reduce phase, orchestrated by MPI.

- **Map Phase:** Each worker process receives a subset of the total data points. In the `map_and_combine` function, each worker iterates through its local points, assigning each one to the nearest of the current K centroids. Most importantly, this phase includes a "local combine" optimization: each worker pre-aggregates the sum of coordinates and the count of points for each cluster among its local data. This reduces the amount of data that needs to be communicated during the reduce phase.
- **Reduce Phase:** The root process (rank 0) acts as the reducer. It receives the local sums and counts from all worker processes using an `MPI.Reduce` operation with a `SUM` operator. It then computes the new global centroids by dividing the global sums by the global counts.

2.3.1 Data Distribution

1. **Initial Data Scatter:** At the start, the root process reads the complete set of data points and scatters disjoint subsets to all worker processes using `scatter_into_root`.
2. **Centroid Broadcast:** In each iteration, after the root process computes the new centroids, it broadcasts them to all worker processes using `MPI.Bcast`. This ensures all workers have the latest centroids for the next map phase.

2.3.2 Communication Patterns

- `MPI.Scatter`: Used once at the beginning to distribute the input data points.
- `MPI.Bcast`: Used in every iteration to broadcast the updated centroids and the final convergence status.
- `MPI.Reduce`: Used in every iteration to aggregate local sums and counts from workers to the root process to compute new centroids.
- `MPI.Barrier`: Used for synchronization between phases to ensure accurate timing measurements for the map and reduce steps.

2.4 Execution Details

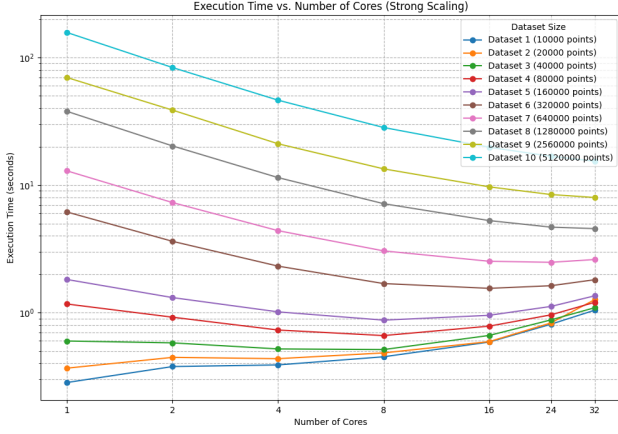
- **Build Command:** The project is built in release mode for performance using Cargo.
`cargo build --release`
- **Execution Command:** The compiled binary is executed with `mpirun`, providing several command-line arguments.

```
mpirun -np <num_procs> ./target/release/kmeans_mapreduce <num_centers> <points_file> <
```

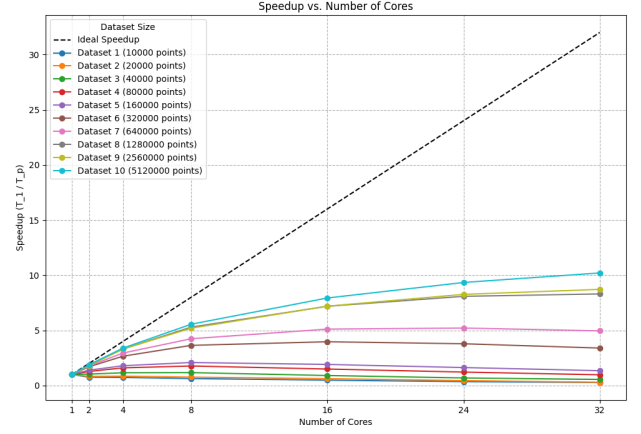
- `<num_procs>`: The number of MPI processes to use.
- `<num_centers>`: The number of clusters to find (K).
- `<points_file>`: Path to the file containing data points.
- `<centers_file>`: Path to the file with initial cluster centers.
- `<max_iter>`: The maximum number of iterations to perform.
- `<out_dir>`: The directory where output files will be saved.

2.5 Performance Evaluation

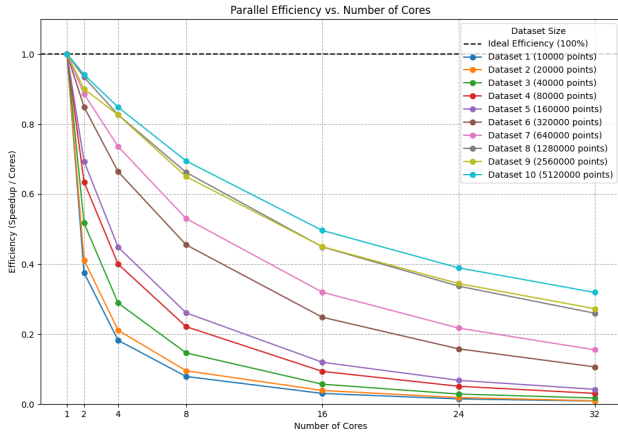
Performance was evaluated to show how the implementation scales with the number of cores.



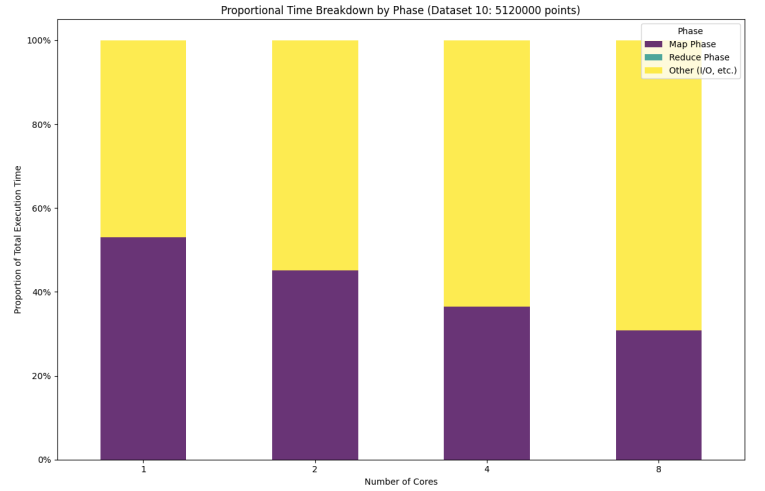
(a) Execution Time vs. Number of Cores



(b) Speedup vs. Number of Cores



(c) Efficiency vs. Number of Cores



(d) Proportional Breakdown for Dataset 10

Figure 3: Performance Analysis: (a) Clear reduction in execution time with increasing cores. (b) Near-linear speedup at lower core counts with diminishing returns due to communication overhead. (c) High parallel efficiency that decreases with more cores. (d) Time distribution across map, reduce, and other phases for a large dataset.

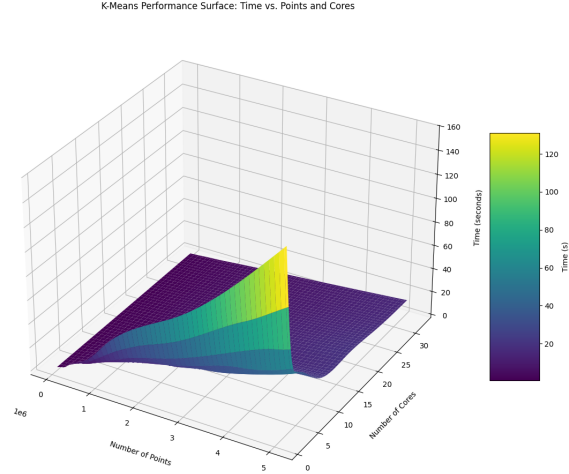


Figure 4: 3D Performance Surface. This plot shows the relationship between the number of cores, the size of the dataset, and the execution time.

2.6 Challenges and Limitations

- **Iterative Communication Overhead:** The algorithm requires a broadcast and a reduce operation in every single iteration. For datasets that require many iterations to converge, this communication can become a significant performance bottleneck, limiting scalability.
- **Root Process Workload:** The root process is solely responsible for computing the new centroids and checking for convergence. While not computationally intensive, this serial step is part of Amdahl's law and limits overall speedup.
- **Memory Constraints:** The initial scatter requires the root process to first load the entire dataset into memory, which can be a limitation for extremely large datasets.

2.7 Conclusion and Future Work

The K-Means algorithm was successfully implemented using an MPI-based MapReduce model in Rust. The use of a local combiner significantly optimizes the reduce step, and the overall implementation shows good scalability.

Potential future work includes:

- Implementing a tree-based reduction (`MPI_Allreduce`) so that all processes have the new centroids without waiting for a broadcast from the root, potentially reducing communication latency.
- Exploring parallel I/O to allow all processes to read their portion of the data directly from the filesystem.
- Implementing a more advanced version of K-Means, such as K-Means++, to improve the quality of the initial centroid selection.

3 gRPC Matrix Calculation Server

3.1 Problem Understanding

This project is a gRPC server, written in Rust, that provides a service for matrix operations. The server is designed to allow a client to build a square matrix by sending its rows one by one via a stream. Once the matrix is complete, the server pre-computes and caches its rank and determinant. Subsequent client queries for these properties are answered using the cached values, ensuring high performance for repeated queries.

3.2 System Setup

- **Programming Language:** Rust
- **Frameworks:** `tonic` for gRPC, `tokio` for asynchronous runtime.
- **Protocol:** gRPC with Protocol Buffers.
- **Architecture:** A single-server, multi-client model. The server is asynchronous and can handle multiple client connections concurrently.

3.3 Design and Implementation

The server exposes a `MatrixService` with two RPC methods: `Interact` and `Reset`.

3.3.1 Concurrency and State Management

The server needs to manage a shared state (the matrix being built, its dimensions, and cached results) across multiple concurrent client requests. This is achieved using an `Arc<Mutex<AppState>>`.

- **Arc (Atomically Referenced Counter):** Allows multiple threads (or in this case, asynchronous tasks) to safely share ownership of the application state.
- **Mutex (Mutual Exclusion):** Ensures that only one task can access and modify the `AppState` at any given time. This prevents race conditions, such as two clients trying to add rows to the matrix simultaneously.

3.3.2 Communication Patterns

- **Bidirectional Streaming (Interact RPC):** The primary interaction happens through this stream. The client can send a stream of `ClientRequest` messages (e.g., `Row` commands). The server processes these and can send back a stream of `ServerResponse` messages (e.g., query results or errors). This is highly efficient as it uses a single, long-lived HTTP/2 connection.
- **Unary RPC (Reset):** A simple request-response method to clear the server's state and allow for the construction of a new matrix.

3.3.3 Performance Optimization: Caching

The core performance feature is the caching mechanism.

1. The server determines the matrix dimension from the first row it receives.
2. It accepts rows until the number of rows equals the dimension, signifying a complete square matrix.
3. At this exact moment, it triggers the computation of the matrix's rank (via Gaussian elimination) and determinant.
4. These values are stored in `cached_rank` and `cached_determinant` within the `AppState`.
5. All subsequent queries for rank or determinant are answered instantly from these cached values, avoiding any redundant, expensive calculations. The matrix itself is effectively immutable until a `Reset` is called.

3.4 Execution Details

The server and client are run as separate binaries from the same project.

- **Run Server Command:** The server is started using Cargo and listens on ‘`:::1:50051`’.

```
cargo run --bin server
```

- **Run Client Command:** In a separate terminal, the client is run with a path to a matrix file and an optional ID.

```
cargo run --bin client -- <path_to_matrix_file> [client_id]
```

- `<path_to_matrix_file>`: A text file where each line contains a space-separated row of numbers.
- `[client_id]`: An optional string to identify the client in server logs.

3.5 Performance Evaluation

- a) No empirical performance data (e.g., latency benchmarks, throughput tests) was generated for this project.
- b) The design is nevertheless performance-oriented. The primary performance gain comes from the **calculate-once, query-many** caching strategy. The cost of calculating the determinant and rank is paid only a single time when the matrix is finalized. All subsequent queries are $O(1)$ lookups. The use of `tokio` and asynchronous gRPC allows the server to handle many concurrent client connections with high efficiency, as it does not block threads waiting for I/O.

3.6 Challenges and Limitations

- **Mutex Contention:** The use of a single `Mutex` to protect the entire application state is simple and safe, but it can become a performance bottleneck under high load. If many clients attempt to send rows or queries simultaneously, they will be serialized, waiting to acquire the lock.
- **Custom Math Implementation:** The matrix operations (`calculate_rank`, `calculate_determinant`) are custom implementations. While functional, they may not be as numerically stable or as highly optimized as those found in dedicated linear algebra libraries like BLAS or LAPACK.
- **No Partial Results:** The design is rigid: queries can only be answered once the *entire* matrix is built. It does not support queries on partially constructed matrices.

3.7 Conclusion and Future Work

The gRPC server provides a robust and efficient service for its specific use case: building a matrix and repeatedly querying its properties. The use of bidirectional streaming and a caching layer are key strengths of the design.

Future work could involve:

- **Improved Concurrency:** Replacing the single `Mutex` with more granular locking (e.g., `RwLock` to allow concurrent reads) could improve performance for query-heavy workloads.
- **Production-Grade Math Library:** Integrating a battle-tested linear algebra library could improve the performance and numerical stability of the rank and determinant calculations.
- **Benchmarking:** Developing a proper benchmarking suite to measure request latency, throughput, and scalability under various client loads.